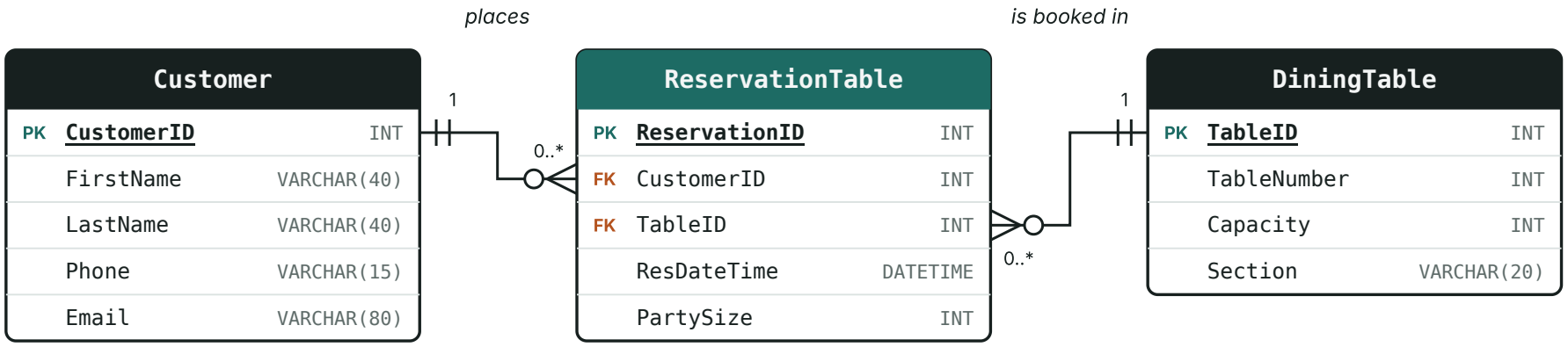
CSC 4500 · HW3 · RESTAURANT RESERVATIONS · LOGICAL MODEL

Crow's foot ERD

The Chen diagram's M:N relationship is resolved into a bridge table, `ReservationTable`. Each box below is one MySQL table, and each line is a foreign key.



UNIQUE (TableID, ResDateTime): a table can't be booked twice for the same time

`Customer` → `ReservationTable` (1 to 0..many)

- A customer makes zero or many reservations.
- Each reservation belongs to exactly one customer.

`DiningTable` → `ReservationTable` (1 to 0..many)

- A dining table appears in zero or many reservations.
- Each reservation is for exactly one dining table.

Data dictionary

Use this when building the CSV files: each CSV's columns match one table, in this order. Leave the AUTO_INCREMENT ID out of the CSV or fill it with 1, 2, 3...

Customer	TYPE	NULL	KEY	RULE / EXAMPLE
CustomerID	INT AUTO_INCREMENT	No	PK	1, 2, 3...
FirstName	VARCHAR(40)	No		Maria
LastName	VARCHAR(40)	No		Lopez (indexed for name search)
Phone	VARCHAR(15)	No		314-555-0142 (text, not a number)
Email	VARCHAR(80)	Yes	UNIQUE	mlopez@example.com

DiningTable	TYPE	NULL	KEY	RULE / EXAMPLE
TableID	INT AUTO_INCREMENT	No	PK	1, 2, 3...
TableNumber	INT	No	UNIQUE	The number painted on the table, e.g. 12
Capacity	INT	No		CHECK 1–20 seats
Section	VARCHAR(20)	No		CHECK one of Main, Patio, Bar

ReservationTable	TYPE	NULL	KEY	RULE / EXAMPLE
ReservationID	INT AUTO_INCREMENT	No	PK	1, 2, 3...
CustomerID	INT	No	FK	→ Customer.CustomerID · ON DELETE CASCADE
TableID	INT	No	FK	→ DiningTable.TableID · ON DELETE RESTRICT
ResDateTime	DATETIME	No		2026-11-20 19:30:00 (format MySQL expects in the CSV)
PartySize	INT	No		CHECK > 0; should not exceed the table's Capacity (check this in the Java form)

Crow's foot symbol key



CREATE TABLE script

```sql
-- MySQL 8.0.16+ (enforces CHECK constraints)
CREATE DATABASE IF NOT EXISTS restaurant_db;
USE restaurant_db;

CREATE TABLE Customer (
    CustomerID   INT          AUTO_INCREMENT PRIMARY KEY,
    FirstName    VARCHAR(40)  NOT NULL,
    LastName     VARCHAR(40)  NOT NULL,
    Phone        VARCHAR(15)  NOT NULL,
    Email        VARCHAR(80)  UNIQUE
);

CREATE TABLE DiningTable (
    TableID      INT          AUTO_INCREMENT PRIMARY KEY,
    TableNumber  INT          NOT NULL UNIQUE,
    Capacity     INT          NOT NULL CHECK (Capacity BETWEEN 1 AND 20),
    Section      VARCHAR(20)  NOT NULL
                 CHECK (Section IN ('Main', 'Patio', 'Bar'))
);

CREATE TABLE ReservationTable (
    ReservationID INT         AUTO_INCREMENT PRIMARY KEY,
    CustomerID    INT         NOT NULL,
    TableID       INT         NOT NULL,
    ResDateTime   DATETIME    NOT NULL,
    PartySize     INT         NOT NULL CHECK (PartySize > 0),
    CONSTRAINT fk_res_customer FOREIGN KEY (CustomerID)
        REFERENCES Customer (CustomerID) ON DELETE CASCADE,
    CONSTRAINT fk_res_table FOREIGN KEY (TableID)
        REFERENCES DiningTable (TableID) ON DELETE RESTRICT,
    CONSTRAINT uq_table_time UNIQUE (TableID, ResDateTime)
);

CREATE INDEX idx_res_datetime      ON ReservationTable (ResDateTime);
CREATE INDEX idx_customer_lastname ON Customer (LastName);
```

Notes for the SQL portion

1. **Order matters.** Create and load Customer and DiningTable first. ReservationTable comes last because its foreign keys must point at rows that already exist.
2. **Why "DiningTable"?** `TABLE` is a reserved word in MySQL, so a table named "Table" would need backticks everywhere.
3. **CASCADE vs RESTRICT.** Deleting a customer removes their reservations. Deleting a dining table that still has bookings is blocked, so history isn't lost by accident.
4. **No double-booking.** `uq_table_time` rejects a second booking for the same table at the same time. Your Java booking form should catch that error and show a friendly message.
5. **Party size vs capacity.** MySQL can't CHECK across two tables, so validate `PartySize ≤ Capacity` in the Java insert form. It also works as a query: find reservations that overfill their table.
6. **Who builds what.** Following your checklist: A owns Customer, B owns DiningTable, C owns ReservationTable plus the keys and indexes.